Smishing Triage — Technical Writeup

BC3415 AI in Accounting & Finance · Individual Assignment (Option A) · @Jonathan · Oct 12, 2026

1. Problem and business case

Smishing Triage is a three-way message classifier that separates legitimate messages (**ham**), commercial **spam** and **smishing** (phishing by SMS). It is delivered as a forward-it-to-check Telegram bot for Singapore users. On held-out test data it catches 89% of smishing messages and wrongly flags 0.26% of real Singaporean messages.

Scams cost people in Singapore about S$913.1 million across 37,308 cases in 2025, down from S$1.11 billion in 2024 (SPF Annual Scam and Cybercrime Brief 2025). Losses were still S$410.6 million in the first half of 2026 alone (SPF Mid-Year Brief 2026). Telcos and banks filter at the network level, but messages still get through, and a user has no quick way to check one specific message.

The three-way split is the point. A standard ham-versus-spam filter puts a marketing blast and a credential-harvesting message in the same class. For the user, one wastes three seconds and the other can empty a bank account. Separating them turns an inbox tidier into a fraud control, so the headline metric is **recall on smishing**, not accuracy: on a corpus that is 83% ham, a model that calls everything ham scores 83% accuracy and catches nothing.

2. Data

Only one public corpus separates commercial spam from smishing, and it holds just 638 smishing messages. That class is the project's bottleneck. Four corpora were combined; their roles are fixed so that no source can leak into the wrong place.

Source	Messages (raw)	After deduplication	Role
Mendeley SMS Phishing	5,971 (4,844 ham, 489 spam, 638 smishing)	5,823	Core three-class set; the only real source of smishing
UCI SMS Spam Collection	5,574 (ham, spam)	184	Extra ham and spam; almost all of it is already inside Mendeley

Source	Messages (raw)	After deduplication	Role
Mendeley Balanced Spam/Smishing	10,191	3,302	LLM-generated spam and smishing; **training only, never evaluated on**
NUS SMS Corpus (Singapore senders)	10,116	8,948	Singaporean ham; held out to measure false alarms on local text

Deduplication removed 13,595 of 31,852 rows. The corpora are built on each other: Mendeley re-labels UCI, and every ham row in the balanced set duplicates real Mendeley ham. What survives from the balanced set is purely generated spam (1,344) and smishing (1,958), which makes up 84% of the training smishing class.

A second, hand-collected set of real scam texts sent to people in Singapore is used for evaluation only (section 5).

3. Preprocessing pipeline

The pipeline's job is to make every evaluation number honest: no message, and no variant of a message, may appear on both sides of a split. It runs in five steps (`src/preprocess.py`).

1. **Load and unify.** One loader per corpus emits the same three columns, `text`, `label` and `source`. All files are read as UTF-8; reading them as latin-1 had turned every "£" into "Â£" in over a thousand training messages, a cue no real incoming message carries.
2. **Deduplicate before splitting.** Messages are matched on a key that ignores case, spacing and punctuation. When copies disagree, the finer label wins (Mendeley over UCI). The 303 conflicts are almost all `uci=spam` against `mendeley=smishing` on identical text: the spam/smishing conflation this project exists to undo.
3. **Normalise, keeping two representations.** `text_clean` replaces URLs, emails, phone numbers (including 8-digit Singapore numbers), amounts and numbers with placeholder tokens, because their presence is predictive even when the value is not. The raw `text` is kept for character features, since digits, case and punctuation are what obfuscation abuses (`DB5`, `acc0unt`).
4. **Split by template, not by row.** Scam campaigns send one template with a different phone number or prize amount each time, and exact deduplication keeps every variant. A row-level split put 27% of test smishing beside a near-identical sibling in training. Messages are therefore grouped by their normalised text and each group is assigned to one split, stratified by label (80/20, then 85/15 for validation).

5. **Bar synthetic and local data from the wrong places.** LLM-generated rows join training only; evaluating on them would measure the generator. Singaporean ham becomes a separate shift set. Any synthetic or shift row sharing a template across a boundary is dropped.

Split	Messages	Ham	Spam	Smishing
Train	7,383	3,375	1,684	2,324
Validation	735	595	61	79
Test	1,191	992	98	101
Singapore shift (NUS)	8,942	8,942	0	0

With 101 smishing messages in test, one misclassification moves recall by a full point, so smishing recall is always reported with a bootstrap confidence interval.

4. Models and selection

A character n-gram SVM won on validation with macro F1 0.945, ahead of three cheaper and simpler models. Four models were fitted, cheapest first, and each was scored on validation only; the test set was used once, by the winner.

Model	Features	Validation macro F1	Smishing recall	Smishing precision
Multinomial Naive Bayes	word counts	0.900	0.924	0.839
Logistic regression	word TF-IDF (1–2 grams)	0.926	0.911	0.911
Logistic regression	word + character TF-IDF	0.928	0.899	0.922
Calibrated linear SVM	**character TF-IDF (2–5 grams), case kept**	**0.945**	**0.949**	**0.915**

- **Selection rule.** Highest validation macro F1, with smishing recall as the tie-break. It is applied in code, not by eye.
- **Class imbalance.** Every model except Naive Bayes uses balanced class weights, so the loss counts a missed smishing message as heavily as the class is rare.

- **Why characters win.** Smishing hides in spellings and link shapes that word tokens destroy. Keeping case added a further point of macro F1 (0.936 to 0.945), because shouted calls to action and brand spellings are signal.
- **Calibration.** The SVM is wrapped in sigmoid calibration so it outputs probabilities. The bot needs a confidence to show, and the operating-point comparison needs them too.

Operating point. The model outputs three probabilities, and a decision rule turns them into one verdict. *Argmax* takes the most probable class. *Min-cost* takes the class with the lowest expected cost under the project's cost matrix, where a missed scam costs 20 and a false scam alarm costs 2. Both were compared on validation and the Singapore shift set before the test set was touched.

Rule	Macro F1	Smishing recall	Smishing precision	False alarms on Singapore ham
Argmax (served)	0.945	0.949	0.915	0.26%
Min-cost	0.891	0.987	0.780	1.77%

The cost matrix prefers min-cost. But it buys three extra caught scams with a sevenfold rise in false alarms on real local messages, a cost the matrix has no term for. For a checker people must trust, the served rule is argmax, set in `config.yaml`. The finding is reportable in itself: a cost matrix is only as good as the costs it leaves out.

5. Results

On the held-out test set the model catches 89.1% of smishing (95% CI 82.6–94.7%) at 85.7% precision, and wrongly flags 0.26% of real Singaporean messages. On real scams sent to people in Singapore it catches 63%, and that gap is the project's most important finding.

Test metric	Value
Smishing recall	**0.891** (95% CI 0.826–0.947)
Smishing precision	0.857
Macro F1	0.904
Accuracy	0.973
False alarms on Singaporean ham (NUS)	0.26% (23 of 8,942)

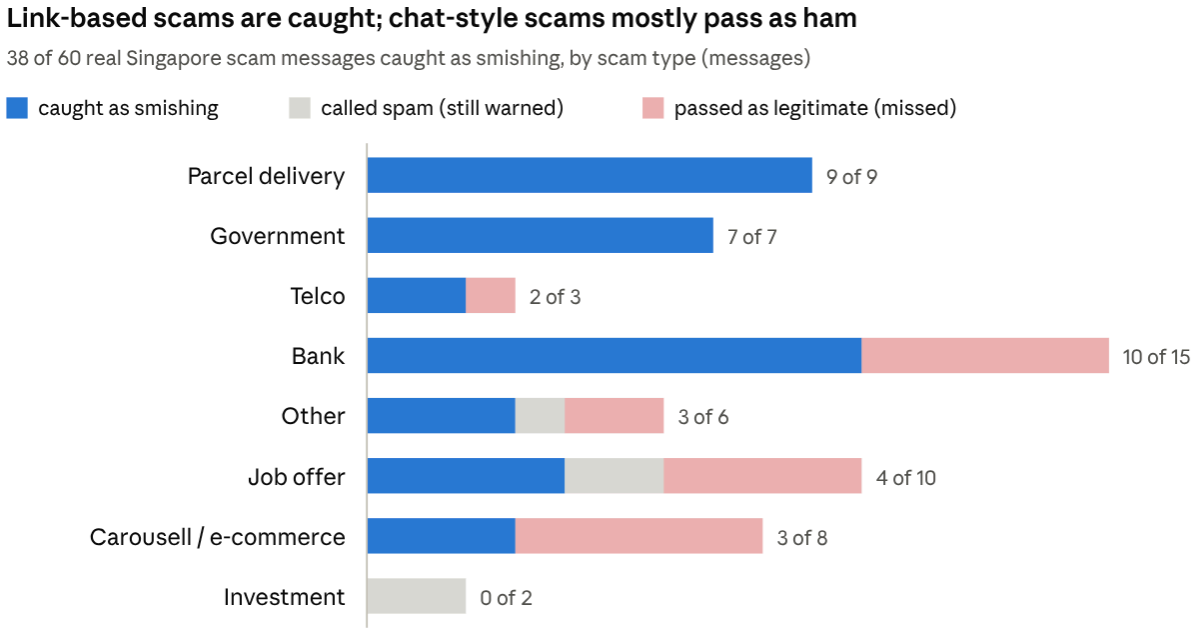
True ↓ / predicted →	Ham	Spam	Smishing
Ham (992)	989	1	2
Spam (98)	5	80	13
Smishing (101)	0	11	90

Almost all error sits on the spam/smishing boundary, and no smishing message passed as legitimate. A scam called spam still reaches the user with a warning, so the costly miss (a scam marked safe) did not occur in test. An earlier row-level split reported smishing recall of 0.954, but a quarter of its test smishing had a near-identical sibling in training; 0.891 is the honest figure.

Synthetic augmentation earns its place. Training with the LLM-generated messages raised validation smishing recall from 0.911 to 0.949, at the cost of five more false alarms in 8,942 Singaporean messages.

Real Singapore scams

To measure misses on local scams, 60 scam messages and 5 legitimate messages sent to people in Singapore were collected from Singapore Police Force advisories (45 scams, all 5 legitimate), Mothership (8) and archived ScamAlert stories (9), dated 2018 to 2026. Most sources publish these as screenshots, so 51 of the 60 scam texts were transcribed by hand, and each is marked so in the file. The set was scored once and never used for training or tuning.



reports/sg_scams_predictions.csv · 60 scam messages, 2018–2026

- **The model learned what a scam SMS with a link looks like.** Scams carrying a link were caught 29 times out of 32; scams without one, 9 out of 28. Parcel and government impersonation, almost all link-based, were caught every time.
- **Conversational scams pass as chat.** Carousell fake buyers, job recruiters and a kidnap threat read like personal messages, and personal messages are what the training ham is made of.
- **Legitimate local transactional messages are flagged.** All 5 genuine SingPost notices and bank OTPs were called spam or smishing. The training ham is almost entirely personal chat, so an official OTP looks more like a scam than like ham. Five messages is a small sample, but the direction is clear.

Version 2 (future work). The fix is local training data: conversation-stage scams labelled smishing, and genuine transactional messages from banks and SingPost labelled ham. It was not pursued in this version because the public data for these message types is very small compared with the corpora in section 2. Most local scam examples are published only as screenshots, many chat scams circulate in Chinese, and genuine bank notifications are rarely published at all. Any such data would also have to be collected separately from this Singapore set, which must stay unseen to remain a fair test.

6. Explainability

Every spam or scam verdict comes with the words that drove it, found by LIME and shown to the user as red flags. A user who sees *why* a message was flagged learns to spot the next one unaided, which makes the explanation part of the product, not just a report figure.

LIME deletes random subsets of words from the message, re-scores each variant with the real model, and fits a small weighted linear model to those scores. Each word's weight is its estimated push towards the verdict. Deleting many words at once is what lets it explain a 95%-confident prediction; an earlier leave-one-out probe failed there, because removing any single word moved nothing.

Message	Verdict	Red flags (LIME)
Your DBS account is locked. Verify at http://bit.ly/x9	Likely scam (94%)	`http://bit.ly/x9`, account, Verify
URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010	Likely scam (87%)	£100,000, URGENT, Prize

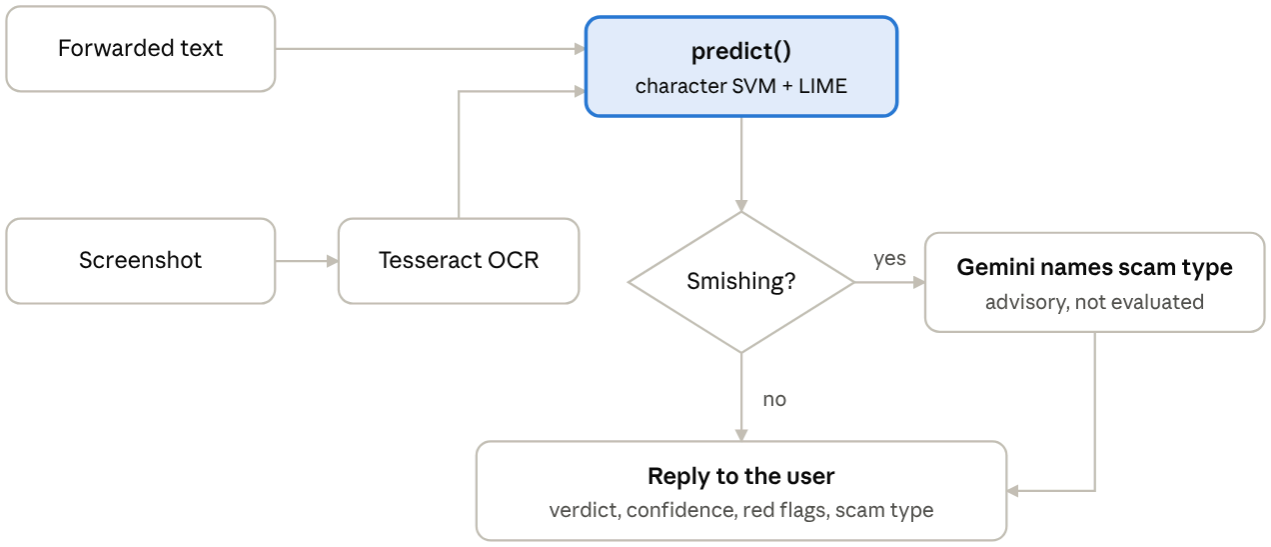
Message	Verdict	Red flags (LIME)
WIN a FREE iPhone! Txt WIN to 85233 now. T and Cs apply	Marketing / spam (92%)	Txt, FREE, now, 85233
Eh you reaching already? I order first ah	Looks legitimate (99%)	none: legitimate messages carry no flags

Tokens are split on whitespace so a link stays whole and is shown as one flag. Explanations are seeded, so the same message always yields the same flags, and they take under half a second. LIME explains the model's behaviour, not ground truth: a common word such as "Your" can appear when it co-occurs with the real cue.

7. The application

The product is a forward-it-to-check Telegram bot: a user forwards a suspicious message, or a screenshot of one, and gets a verdict, a confidence and the red flags back within seconds. Telegram was chosen because WhatsApp is end-to-end encrypted with no read API; forwarding is the standard workaround, and is how Singapore's CheckMate bot works.

The trained model decides; the LLM only names the scam type



serving path · app/bot.py, src/ocr.py, src/predict.py, src/scam_type.py

Only the accented step is evaluated. The trained classifier makes every decision. A zero-shot Gemini call runs **only** on messages the classifier already called smishing, and adds a likely scam type (parcel, bank impersonation, job offer…) with the phrases that indicate

it. Scam-type labels do not exist as public labelled data, so this stage is explicitly advisory and no metric depends on it. The gate lives inside `scam_type.enrich()`, not in the caller, and a test checks that a legitimate message never reaches it.

- **Screenshots.** Users forward screenshots more readily than they retype. Tesseract reads the image, chat furniture (timestamps, "Delivered") is stripped, and the text goes through the same `predict()`.
- **Privacy.** The bot logs a label, a confidence and a message length, never the message body. Legitimate messages never leave the machine. Flagged messages do go to Google, and on the free tier Google may use them to improve its models; a billed account or a locally hosted model would close that gap.
- **Degrades, never fails.** Without a Gemini key the bot still answers with the classifier's verdict; without Tesseract it asks the user to paste text instead.

8. Limitations, risks and scope

The largest risk is that the training smishing is not Singaporean: it is US, UK and Indian in phrasing and brand names, and no public local smishing corpus exists. Section 5 measures what that costs on real local scams.

- **Small smishing class.** 546 real smishing messages survive deduplication, and the test set holds 101. The 95% confidence interval on test recall is six points either side.
- **Synthetic augmentation.** 84% of training smishing is LLM-generated. The ablation shows it helps on real messages, but the model may still have learned some of the generator's style.
- **Concept drift.** Scam wording changes faster than any static training set. A deployment would need periodic retraining on newly reported scams and monitoring of the verdict mix.
- **Adversarial evasion.** Attackers obfuscate deliberately. Character n-grams help with spellings like `DB5`, but a determined attacker can still write a message the model has not seen.
- **OCR errors** land upstream of the classifier. A misread character becomes a feature the model never trained on; dark mode and low resolution make this worse.
- **The spam/smishing boundary** holds almost all remaining error. Some source labels on that boundary are themselves debatable (premium-rate prize texts are labelled smishing in Mendeley and spam in UCI).
- **Privacy.** Flagged messages are sent to Google for the scam-type label (section 7). A scam checker that ships users' messages to a third party has a tension in it, which should be stated to users.

Out of scope, deliberately: email phishing (headers, SPF/DKIM, attachments are a different feature space), phishing website detection, voice and in-person scams, live WhatsApp interception (no read API), and automatic on-device SMS filtering.

9. Reproducing the results

Every number in this writeup is regenerated by one command, `python main.py --skip-download`, from the data in `data/raw/`. All settings (labels, paths, split ratios, seed, cost matrix, operating point) live in `config.yaml`; no module hardcodes a path or a label.

Command	What it does
`python -m src.download`	Fetches UCI, NUS and the OCR language data; lists the two Mendeley files that must be downloaded by hand
`python -m src.preprocess`	Deduplicates, normalises and writes the template-aware splits
`python -m src.train`	Fits the four models, selects on validation, evaluates test once, saves `artifacts/model.joblib`
`python -m src.ablation`	Trains with and without the synthetic data and compares them on validation
`python -m src.predict "message"`	One verdict with confidence and red flags
`python -m src.ocr screenshot.png`	Reads a screenshot, then classifies it
`python -m app.bot`	Runs the Telegram bot (needs `TELEGRAM_BOT_TOKEN`)
`python -m pytest tests`	12 checks, including that no template crosses a split and that the LLM never sees an unflagged message

`notebooks/walkthrough.ipynb` walks through the same pipeline with tables and figures. The code is reusable through one entry point, `predict()`, which returns a label, a confidence, per-class scores and red flags; the bot, the OCR path and the notebook all call it.